

9 Dimensionality Reduction

Engineering data sets often contain many measured variables. A vibration test may produce many frequency-domain features. A manufacturing line may record temperatures, pressures, motor currents, tool speeds, dimensions, and quality measurements. A simulation campaign may output many nodal displacements or stress values. These variables are rarely independent. They often move together because they are driven by a smaller number of physical effects.

Dimensionality reduction replaces a high-dimensional description with a lower-dimensional one. The goal is not to discard information at random. The goal is to find a smaller coordinate system that keeps the structure needed for visualization, preprocessing, compression, denoising, monitoring, or a downstream machine-learning model. Figure 9.1 gives the basic idea.

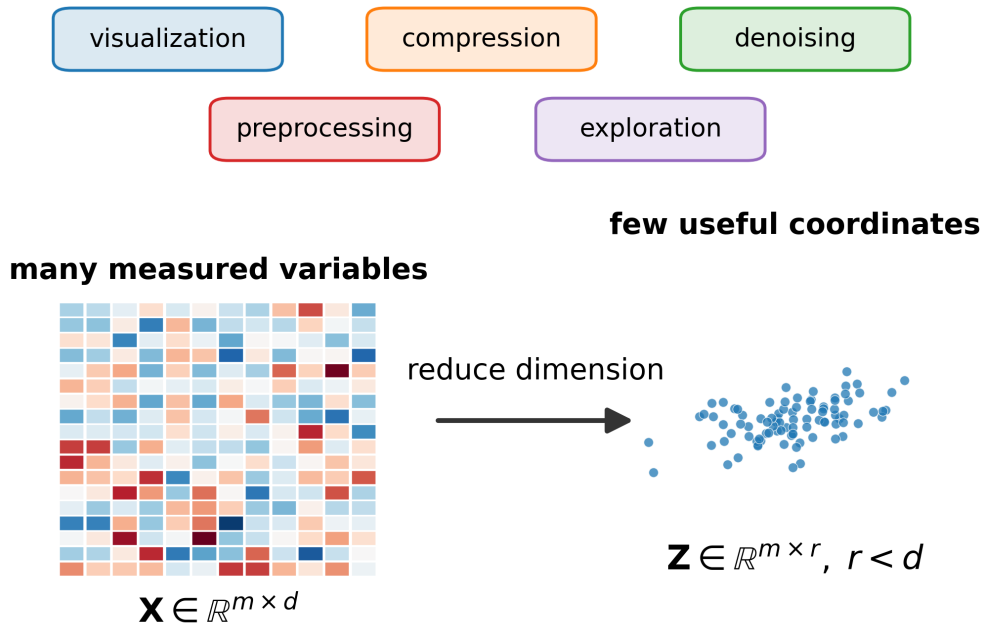


Figure 9.1: Dimensionality reduction converts many correlated measurements into a smaller set of useful coordinates. These coordinates can support visualization, compression, denoising, preprocessing, and exploratory analysis.

The reduced coordinates may be called scores, components, latent variables, embeddings, or reduced features. The words differ by method and by field. The common idea is that each sample is represented with fewer numbers than before.

This chapter uses two running examples. The first is a small synthetic data set, used because its geometry is visible. The second is the Wine data set from `scikit-learn`, used because it is a concrete tabular data set with multiple measured variables and known classes. Figure ?? shows both examples.

The Wine data are not mechanical engineering data, but they behave like many small engineering measurement tables. Each row is a sample. Each column is a numeric measurement. The

columns have different scales and correlations. The samples also belong to known groups, which makes the reduced-coordinate plots easier to interpret.

Table 9.1: Running tabular data set used in several examples.

Quantity	Value
Data source	<code>sklearn.datasets.load_wine()</code>
Samples	$m = 178$
Measured features	$d = 13$ numeric measurements
Known groups	3 classes
Engineering analogy	laboratory measurements, inspection features, process variables, or material descriptors

Notation

- $\vec{x}^{(i)} \in \mathbb{R}^d$: column vector of d measured features for sample i .
- $\mathbf{X} \in \mathbb{R}^{m \times d}$: data matrix with one row per sample and one column per feature.
- m : number of samples.
- d : number of original features.
- r : number of reduced dimensions, with $r < d$.
- $\mathbf{Z} \in \mathbb{R}^{m \times r}$: reduced-coordinate matrix.
- $\bar{\vec{x}}$: sample mean vector.
- $\mathbf{S}$: sample covariance matrix.
- $\vec{u}_j$: the j^{th} principal-component direction.
- λ_j : variance explained by principal component j .

Vectors carry arrows and matrices are bold. Rows of $\mathbf{X}$ are samples and columns of $\mathbf{X}$ are measured variables. This convention matches the mini-batch notation used earlier in the course.

9.1 Why Reduce Dimension?

Dimensionality reduction is used for several related reasons.

- **Visualization:** humans can inspect two or three dimensions much more easily than dozens or thousands.
- **Compression:** a few coordinates can store most of the repeatable variation in a data set.
- **Denoising:** low-variance directions often contain measurement noise rather than stable structure.

- **Preprocessing:** a smaller feature set can make a later model faster, simpler, or more stable.
- **Exploration:** clusters, trends, outliers, and operating regimes may become easier to see.
- **Monitoring:** reduced coordinates can become health indicators, process states, or residual statistics.

Figure 9.2 shows the standardized Wine data matrix. Each row is one sample and each column is one measured feature. The figure makes the dimensionality-reduction problem concrete. The original data live in a 13-dimensional feature space, but we would like to summarize the dominant structure with only a few coordinates.

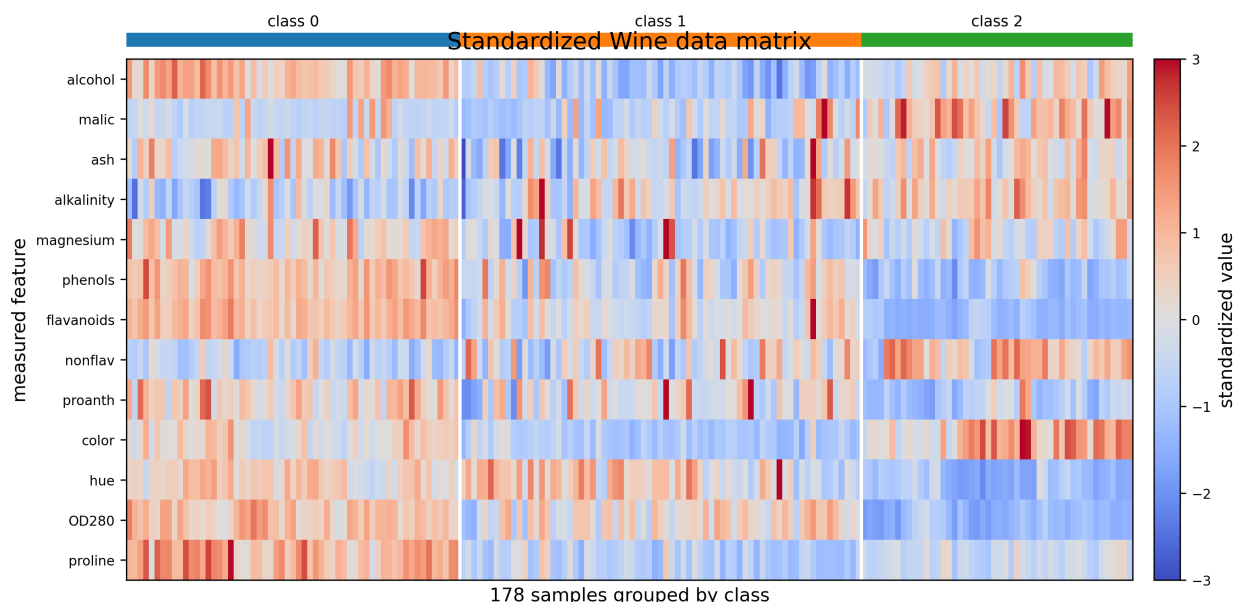


Figure 9.2: Standardized Wine data matrix. Each row is one sample, each column is one measured feature, and the color indicates the standardized value of that measurement. Samples are grouped by their known class labels.

High-dimensional data are difficult for three practical reasons. First, humans cannot directly plot more than two or three coordinates. Second, many machine-learning models need more samples as the number of features grows. Third, distances can become less informative when every sample has many coordinates. This does not mean that high-dimensional data are useless. It means that the data often need to be summarized before interpretation or modeling.

A useful way to think about dimensionality reduction is that it tries to find a simpler coordinate system. In the original coordinate system, a sample might be described by many sensor channels or many measured variables. In the reduced coordinate system, the same sample may be described by a few modes, scores, or embedding coordinates.

The rest of this chapter focuses on methods available in `scikit-learn`. The main method is principal component analysis (PCA). PCA is linear, deterministic, and interpretable. The chapter also introduces t-distributed stochastic neighbor embedding (t-SNE), a nonlinear visualization method, and isometric mapping (Isomap), a graph-based manifold method included in `scikit-learn`.

9.2 Principal Component Analysis (PCA)

Principal component analysis, or PCA, is the most important baseline for dimensionality reduction. PCA finds orthogonal directions along which the data vary the most. The first principal component points in the direction of maximum variance. The second principal component points in the next largest variance direction, subject to being orthogonal to the first. The process continues until all directions are found.

A useful classroom analogy is the shadow analogy. Imagine shining light on a cloud of points and measuring the length of the shadow cast onto a line. If the line is poorly chosen, the shadow is short and many points collapse together. If the line is aligned with the long direction of the data cloud, the shadow is long and preserves more variation. PCA chooses the direction that gives the largest projected variance. Figure 9.3 illustrates this idea.

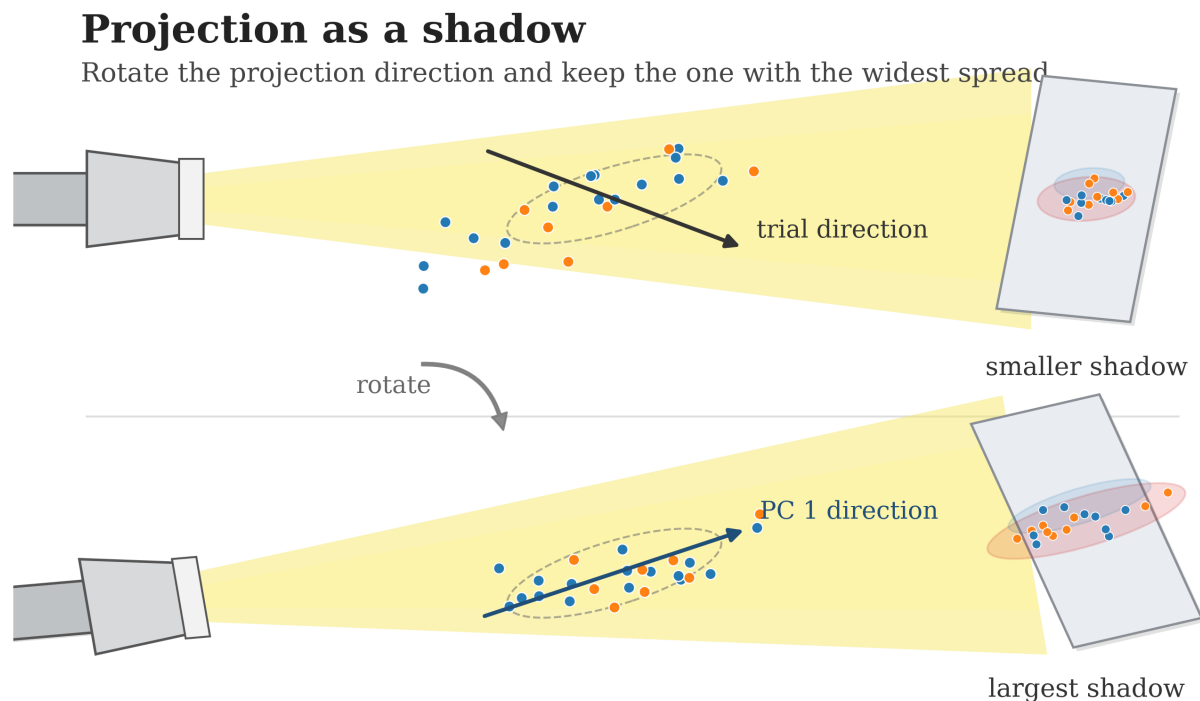


Figure 9.3: Shadow analogy for PCA. Projecting the data onto a weak direction gives a short shadow. Projecting onto the first principal component gives the largest variance shadow and the best one-dimensional linear summary.

Figure 9.4 shows the same idea as a coordinate rotation. The original variables are correlated, so the point cloud is elongated. PCA rotates the coordinate system so that the first new axis lies along the long direction of the cloud.

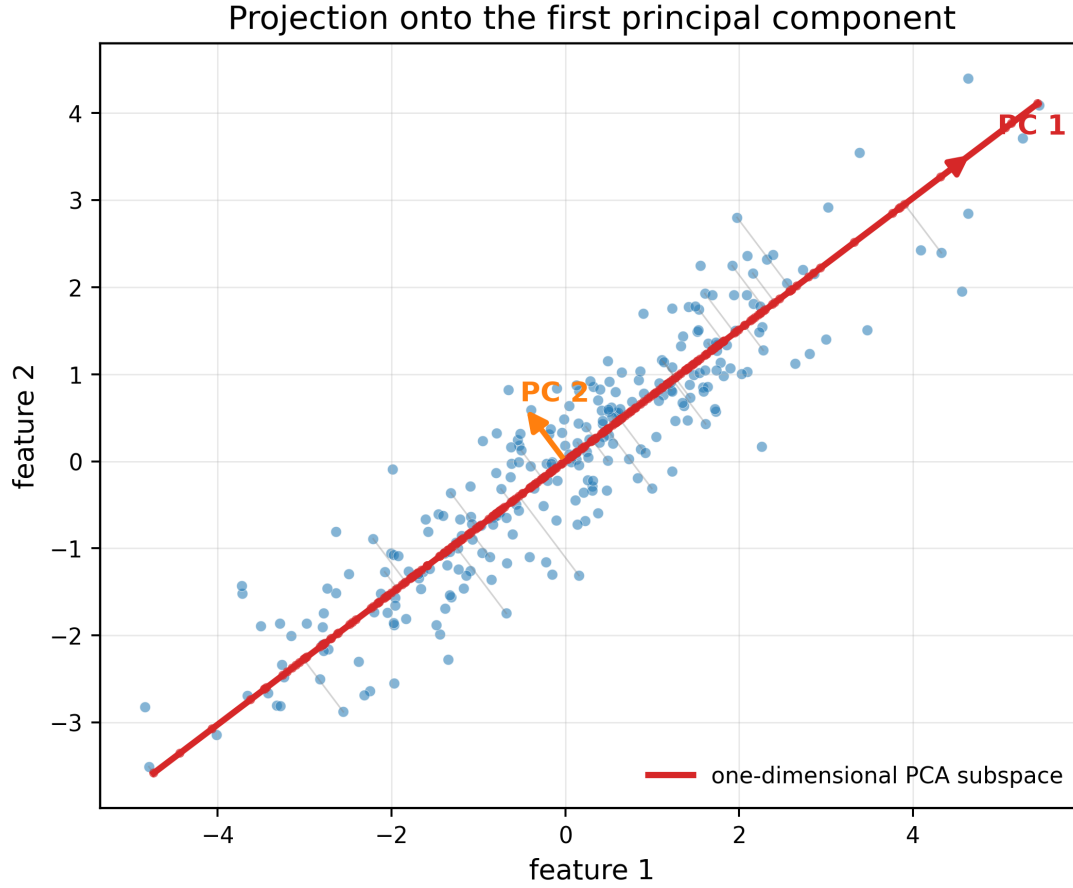


Figure 9.4: PCA rotates the coordinate system toward directions of largest variance. The first principal component captures the largest possible projected variance, and the second captures the next largest orthogonal direction.

9.2.1 Centering the Data

PCA starts by centering the data. Let the mean feature vector be

$$\bar{\vec{x}} = \frac{1}{m} \sum_{i=1}^m \vec{x}^{(i)}. \quad (9.1)$$

The centered data matrix is

$$\mathbf{X}_c = \mathbf{X} - \vec{1}_m \bar{\vec{x}}^\top, \quad (9.2)$$

where $\vec{1}_m$ is a column vector of m ones. The subtraction in Equation (9.2) subtracts the feature mean from every row.

Centering is important because PCA is about variation around the mean. Without centering, the first component can be pulled toward the offset of the data rather than the direction of variation.

9.2.2 Covariance Matrix

After centering, PCA computes the sample covariance matrix

$$\mathbf{S} = \frac{1}{m-1} \mathbf{X}_c^\top \mathbf{X}_c. \quad (9.3)$$

Because $\mathbf{X}_c \in \mathbb{R}^{m \times d}$, the covariance matrix has shape

$$\mathbf{S} \in \mathbb{R}^{d \times d}. \quad (9.4)$$

This is important: the covariance matrix is organized by features, not by samples. If the data matrix has 10 samples and 10 features, the covariance matrix is 10×10 because there are 10 features. If the data matrix has 1000 samples and 10 features, the covariance matrix is still 10×10 .

The entry S_{ij} measures how feature i and feature j vary together across the m samples:

$$S_{ij} = \frac{1}{m-1} \sum_{k=1}^m (X_{ki} - \bar{x}_i) (X_{kj} - \bar{x}_j). \quad (9.5)$$

The diagonal entries are variances because a feature is being compared with itself. The off-diagonal entries are covariances between different features. Since $S_{ij} = S_{ji}$, the covariance matrix is symmetric about the diagonal. Figure 9.5 shows the structure.

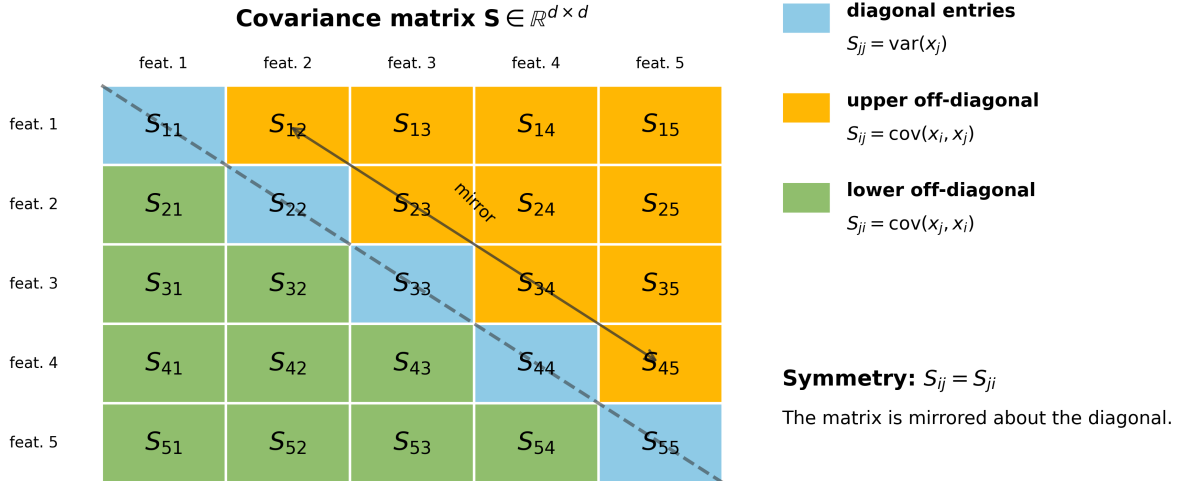


Figure 9.5: Structure of the covariance matrix used by PCA. Diagonal entries are feature variances, off-diagonal entries are covariances between features, and the matrix is symmetric because $S_{ij} = S_{ji}$.

Figure 9.6 shows the covariance matrix for the standardized Wine features. Since the features were standardized, the diagonal entries are one. The off-diagonal colors show which pairs of features tend to move together or in opposite directions.

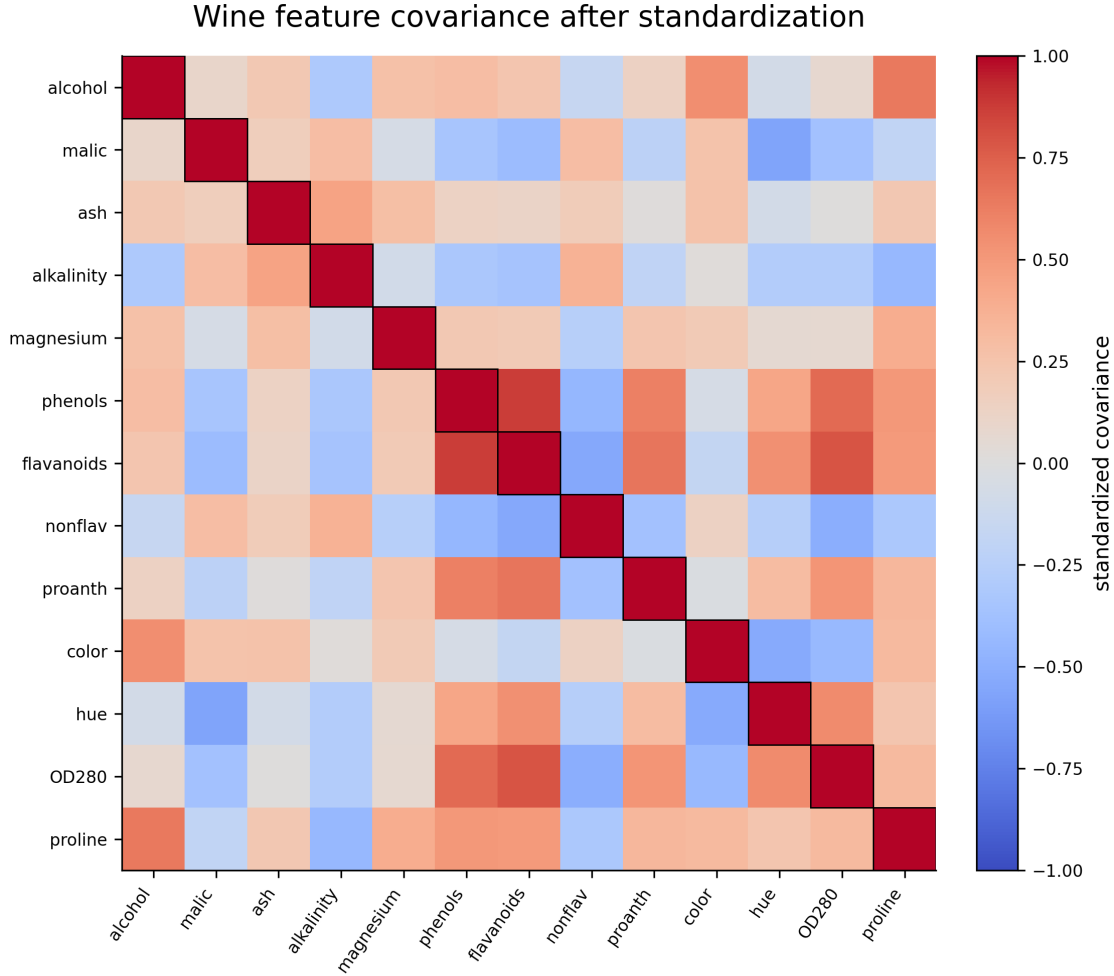


Figure 9.6: Covariance matrix for standardized Wine features. After standardization, the covariance matrix is equivalent to the correlation matrix. The diagonal entries are one, and off-diagonal entries show positive or negative feature relationships.

The covariance matrix is the object that PCA studies. PCA does not take eigenvectors of the raw data matrix $\mathbf{X}$ in this basic covariance-matrix view. Instead, it takes eigenvectors of the covariance matrix $\mathbf{S}$. Those eigenvectors become the principal-component directions.

9.2.3 Eigenvectors and Eigenvalues

PCA solves the eigenvalue problem

$$\mathbf{S}\vec{u}_j = \lambda_j \vec{u}_j. \quad (9.6)$$

The eigenvector $\vec{u}_j$ is a principal-component direction. The eigenvalue λ_j is the variance of the data along that direction. The eigenvalues are usually ordered so that

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0. \quad (9.7)$$

The first eigenvector $\vec{u}_1$ is the first principal component. The second eigenvector $\vec{u}_2$ is the second principal component, and so on.

In practical language, PCA uses the eigenvectors as new axes. The eigenvalues are not the reduced coordinates. Instead, the eigenvalues tell us how much variance is explained by each new axis. We use the eigenvalues to decide how many eigenvectors to keep.

If the first r principal-component directions are kept, collect them in the loading matrix

$$\mathbf{U}_r = [\vec{u}_1 \ \vec{u}_2 \ \cdots \ \vec{u}_r] \in \mathbb{R}^{d \times r}. \quad (9.8)$$

The reduced coordinates, or PCA scores, are

$$\mathbf{Z} = \mathbf{X}_c \mathbf{U}_r \in \mathbb{R}^{m \times r}. \quad (9.9)$$

Each row of $\mathbf{Z}$ is the reduced representation of one sample.

Example 9.1 PCA for a sensor array. Suppose a test fixture produces $d = 80$ features per run. Many features move together because the structure has only a few dominant vibration patterns. PCA may show that the first five components explain most of the variation. Instead of training a later model on all 80 features, an engineer may train it on the first five PCA scores.

9.2.4 Scaling and Standardization

PCA is sensitive to feature scale. If one feature is measured in thousands and another is measured in fractions, the large-scale feature can dominate the covariance matrix even if it is not more physically important. For engineering data, the units of measurement matter.

A common preprocessing step is to standardize each feature:

$$X_{\text{std},ij} = \frac{X_{ij} - \mu_j}{s_j}, \quad (9.10)$$

where μ_j is the mean of feature j and s_j is its sample standard deviation. PCA on standardized data is equivalent to PCA on the correlation matrix rather than the covariance matrix.

Use centered but unscaled data when absolute variance has physical meaning. Use standardized data when variables have different units or very different numerical scales. The choice should be recorded because it changes the components.

9.2.5 Explained Variance and Scree Plots

The explained variance ratio for component j is

$$\rho_j = \frac{\lambda_j}{\sum_{k=1}^d \lambda_k}. \quad (9.11)$$

The cumulative explained variance after keeping r components is

$$R_r = \sum_{j=1}^r \rho_j = \frac{\sum_{j=1}^r \lambda_j}{\sum_{k=1}^d \lambda_k}. \quad (9.12)$$

A scree plot shows the explained variance for each component. Figure 9.7 shows the scree plot for the standardized Wine data. The bars show individual explained variance ratios, and the line shows cumulative explained variance.

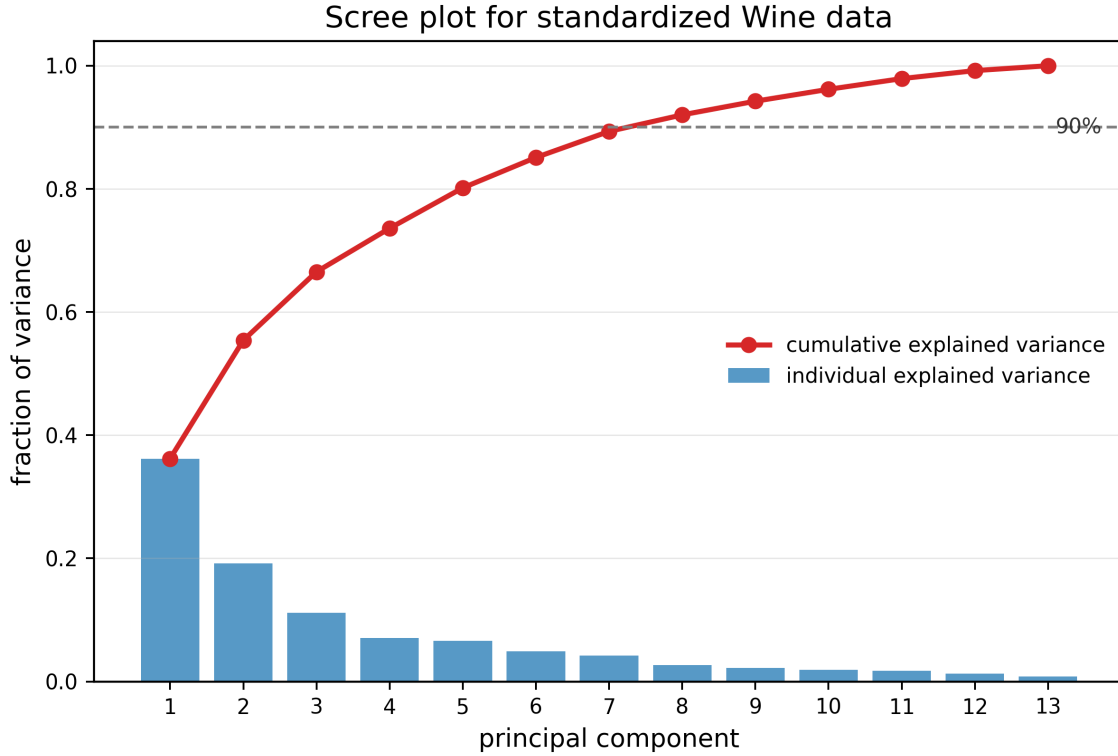


Figure 9.7: Scree plot for the standardized Wine data. Bars show the variance explained by each principal component, and the line shows cumulative explained variance.

A common rule is to keep enough components to explain a selected fraction of variance, such as 90% or 95%. This rule is convenient, but it is not universal. Low-variance components can still matter if they contain fault signatures, rare events, or safety-critical effects. Explained variance should be combined with engineering judgment.

9.2.6 Projection, Scores, and Loadings

Projection is the step that converts original features into PCA scores. For one centered sample $\vec{x}_c$, the score on principal component j is

$$z_j = \vec{u}_j^\top \vec{x}_c. \quad (9.13)$$

The vector of the first r scores is

$$\vec{z} = \mathbf{U}_r^\top \vec{x}_c \in \mathbb{R}^r. \quad (9.14)$$

For a batch of samples, the matrix form is Equation (9.9).

The loadings in $\mathbf{U}_r$ tell how the original variables contribute to the principal components. Large positive or negative loading values indicate that a feature strongly affects that component. Figure 9.8 shows the first three loading vectors for the Wine data.

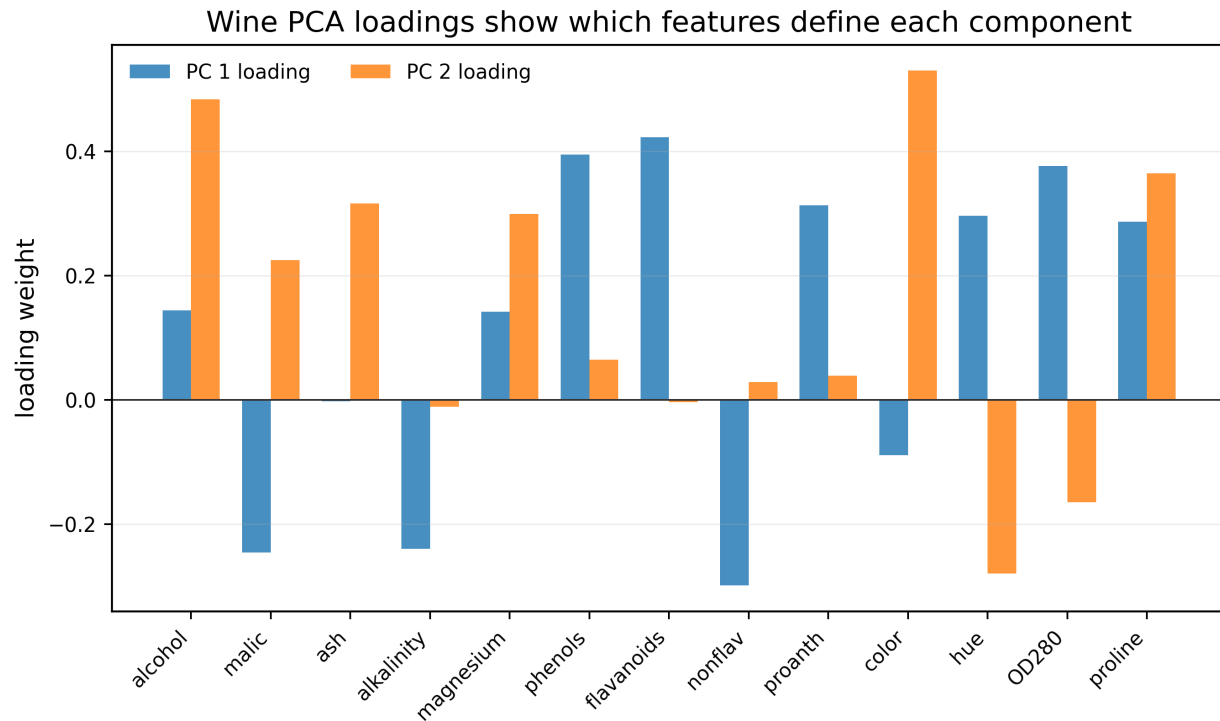


Figure 9.8: PCA loadings for the first three components of the standardized Wine data. Loadings describe how the original measured features contribute to each principal component.

Figure 9.9 shows the Wine samples plotted using the first two PCA scores. The classes were not used to fit PCA, but they are shown in the plot so that the structure can be inspected.

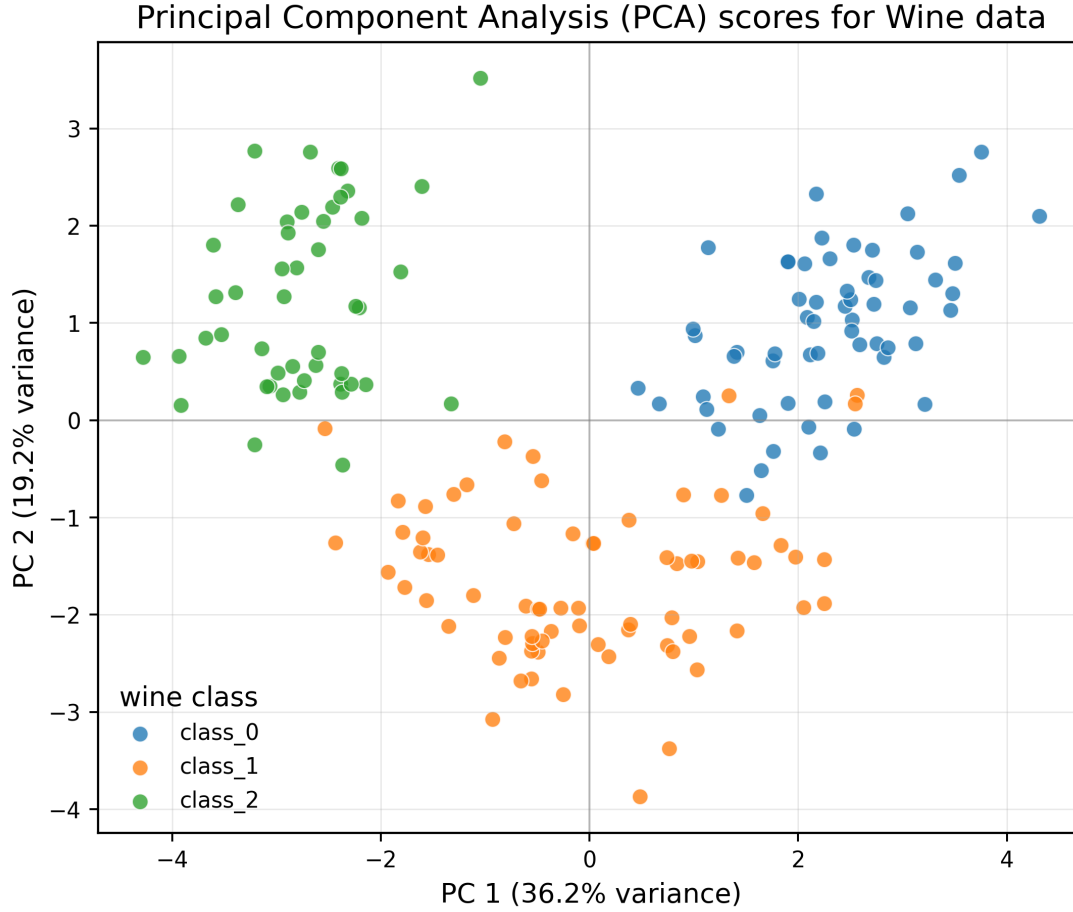


Figure 9.9: PCA score plot for the standardized Wine data. Each point is one sample projected onto the first two principal components. Class labels are shown only for interpretation.

A PCA score plot should be read carefully. If two classes are separated in the first two principal components, that is useful evidence. If they overlap, it does not prove that the classes are inseparable. The class information may lie in lower-variance directions, or the separation may require a nonlinear model.

9.2.7 Reconstruction

PCA can approximately reconstruct the original data from the retained scores. The reconstruction from r components is

$$\hat{\mathbf{X}} = \mathbf{Z}\mathbf{U}_r^\top + \vec{1}_m \bar{\vec{x}}^\top. \quad (9.15)$$

The reconstruction error matrix is

$$\mathbf{E} = \mathbf{X} - \hat{\mathbf{X}}. \quad (9.16)$$

If $r = d$, the reconstruction is exact for centered data. If $r < d$, the reconstruction keeps only the variation captured by the retained components.

Figure 9.10 shows the Wine reconstruction error as the number of retained PCA components increases. More components produce lower reconstruction error, but the cost is a less compact representation.

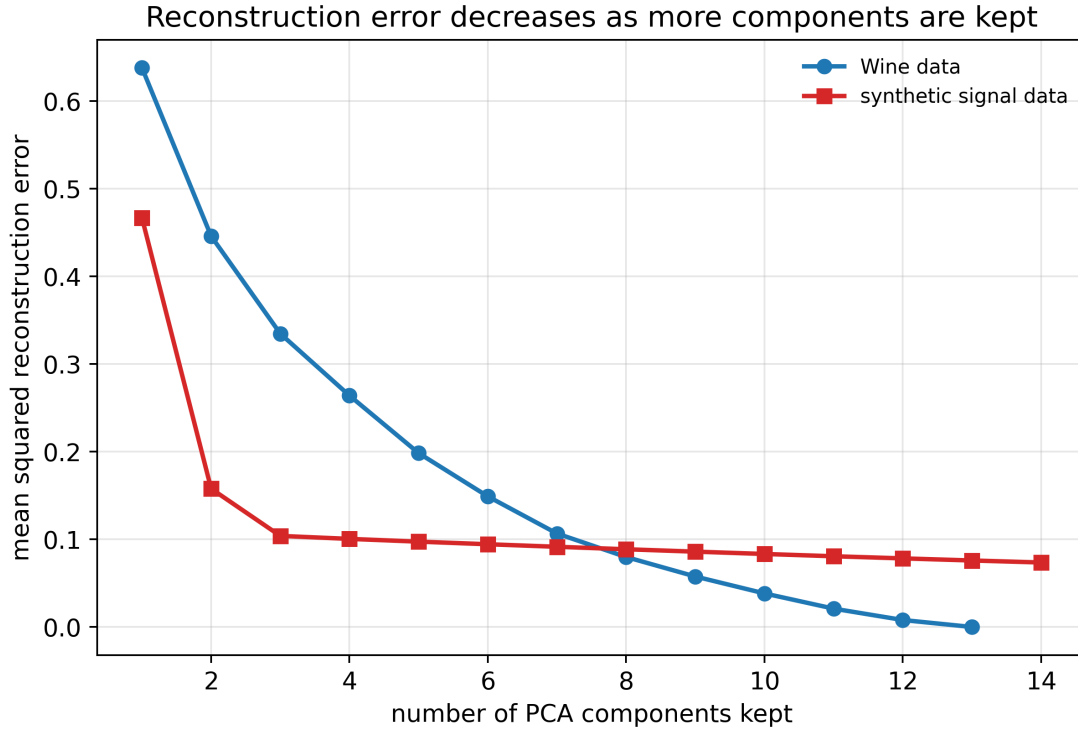


Figure 9.10: PCA reconstruction error for the standardized Wine data. Keeping more principal components lowers the reconstruction error because more directions of variation are retained.

A useful optimization view is that PCA chooses the rank- r linear approximation that minimizes squared reconstruction error:

$$\mathbf{U}_r^* = \arg \min_{\mathbf{U}_r^\top \mathbf{U}_r = \mathbf{I}} \left\| \mathbf{X}_c - \mathbf{X}_c \mathbf{U}_r \mathbf{U}_r^\top \right\|_F^2. \quad (9.17)$$

This is why PCA is often the first baseline for compression and denoising.

9.2.8 Denoising

PCA can remove noise when the repeatable structure lies in a low-dimensional subspace and the noise is spread across many directions. The basic workflow is

$$\mathbf{X} \longrightarrow \mathbf{Z} = \mathbf{X}_c \mathbf{U}_r \longrightarrow \hat{\mathbf{X}} = \mathbf{Z} \mathbf{U}_r^\top + \vec{1}_m \bar{\bar{x}}^\top. \quad (9.18)$$

The low-dimensional scores keep dominant patterns. The reconstruction removes some variation that is not captured by those patterns.

Figure 9.11 shows a synthetic signal example. The noisy measurements are projected into a small PCA space and reconstructed. The reconstruction follows the repeatable structure more closely than the raw noisy signals.

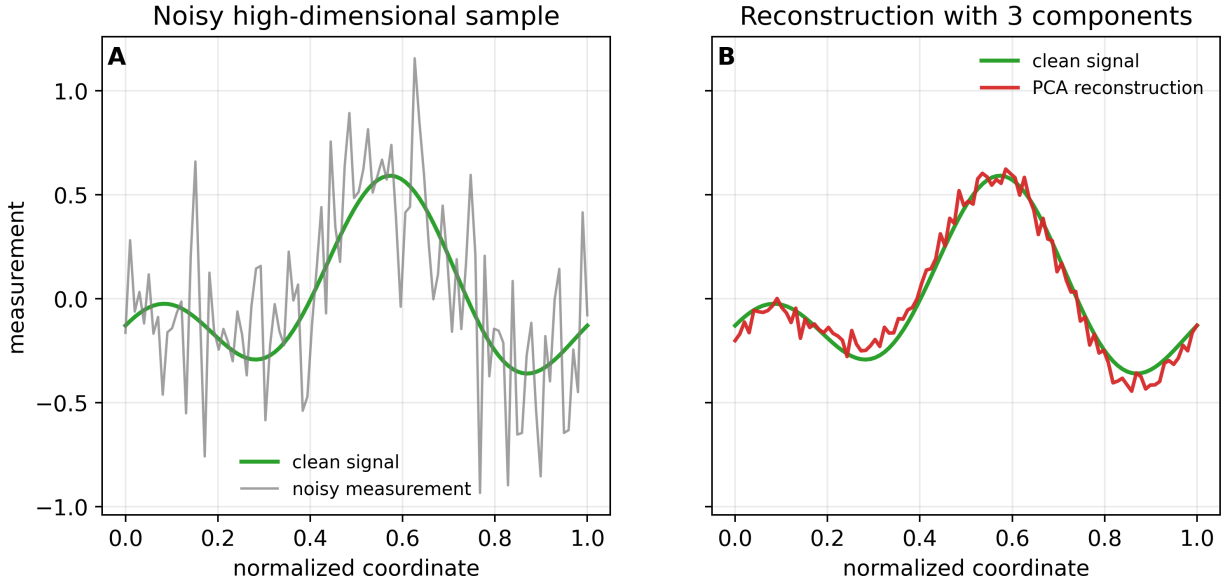


Figure 9.11: PCA denoising by reconstruction. The noisy signals are projected into a small PCA subspace and reconstructed, keeping dominant repeatable structure while suppressing part of the noise.

For engineering data, PCA denoising is useful only when the low-dimensional assumption is reasonable. It can fail if the discarded components contain rare but important damage, faults, or transient behavior.

9.2.9 PCA as a Preprocessing Step

PCA can also be used before a supervised model. The workflow is usually

$$\mathbf{X} \longrightarrow \text{scaling} \longrightarrow \mathbf{Z} = \mathbf{X}_c \mathbf{U}_r \longrightarrow \text{downstream model.} \quad (9.19)$$

The downstream model may be logistic regression, support vector machines, random forests, Gaussian processes, neural networks, or a clustering method.

Figure 9.12 shows a simple Wine classification example. A `scikit-learn` pipeline standardizes the features, projects them onto r PCA components, and trains logistic regression. The figure shows cross-validation accuracy as r changes.

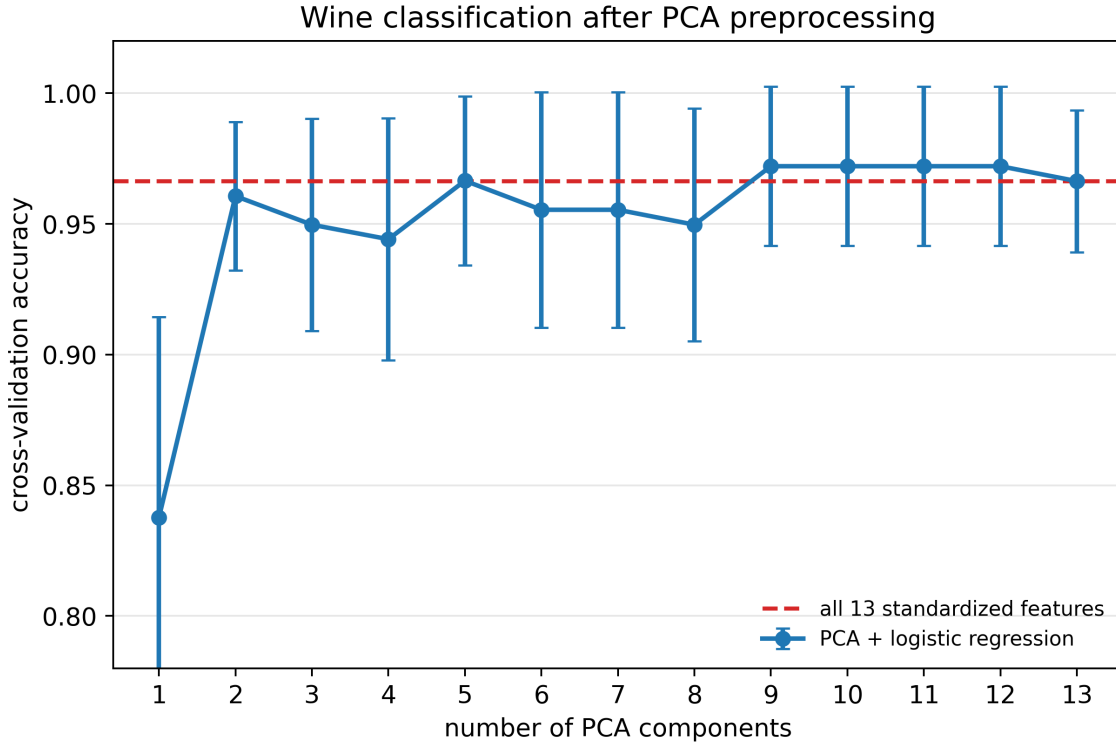


Figure 9.12: Wine classification accuracy after PCA preprocessing. A small number of principal components can be enough for strong performance, but the number of components should be selected using validation data.

For preprocessing, PCA must be fit only on the training data. The same scaling values and principal-component directions are then applied to validation, test, and future samples. Fitting PCA on the full data set before splitting can leak information from the test set into the training process.

Example 9.2 PCA before regression. Suppose the goal is to predict fatigue life from 200 descriptors of a loading history. A small data set makes direct regression unstable. PCA can reduce the descriptors to 10 scores, and the regression model can be trained on those scores. The scaler, PCA model, and regression model must all be fit using training data only.

9.2.10 PCA Residuals and Anomalies

PCA can help anomaly detection by measuring how well a sample is represented by a normal-data subspace. For sample i , define the squared reconstruction residual

$$Q_i = \left\| \vec{x}^{(i)} - \hat{\vec{x}}^{(i)} \right\|_2^2. \quad (9.20)$$

A large value of Q_i means that the sample is not well represented by the retained PCA components. This may indicate a sensor problem, a new operating condition, a data-entry error, or a fault mode.

The residual should not be treated as proof of a specific failure. It is a screening statistic. The engineering interpretation still depends on the sensors, operating conditions, and known failure mechanisms.

9.3 t-distributed Stochastic Neighbor Embedding (t-SNE)

t-distributed stochastic neighbor embedding, usually written t-SNE, is a nonlinear dimensionality-reduction method used mostly for visualization. It tries to place points in two or three dimensions so that nearby points in the original space remain nearby in the embedding.

The important word is nearby. t-SNE is designed to preserve local neighborhoods, not global distances. If two clusters appear far apart in a t-SNE plot, that distance should not be interpreted as a calibrated physical distance. The method is better at showing local groupings than at preserving the full geometry of the original feature space.

Figure 9.13 compares PCA and t-SNE on the standardized Wine data. PCA gives a linear variance-based projection. t-SNE gives a nonlinear local-neighborhood view.

t-Distributed Stochastic Neighbor Embedding (t-SNE) emphasizes local neighborhoods

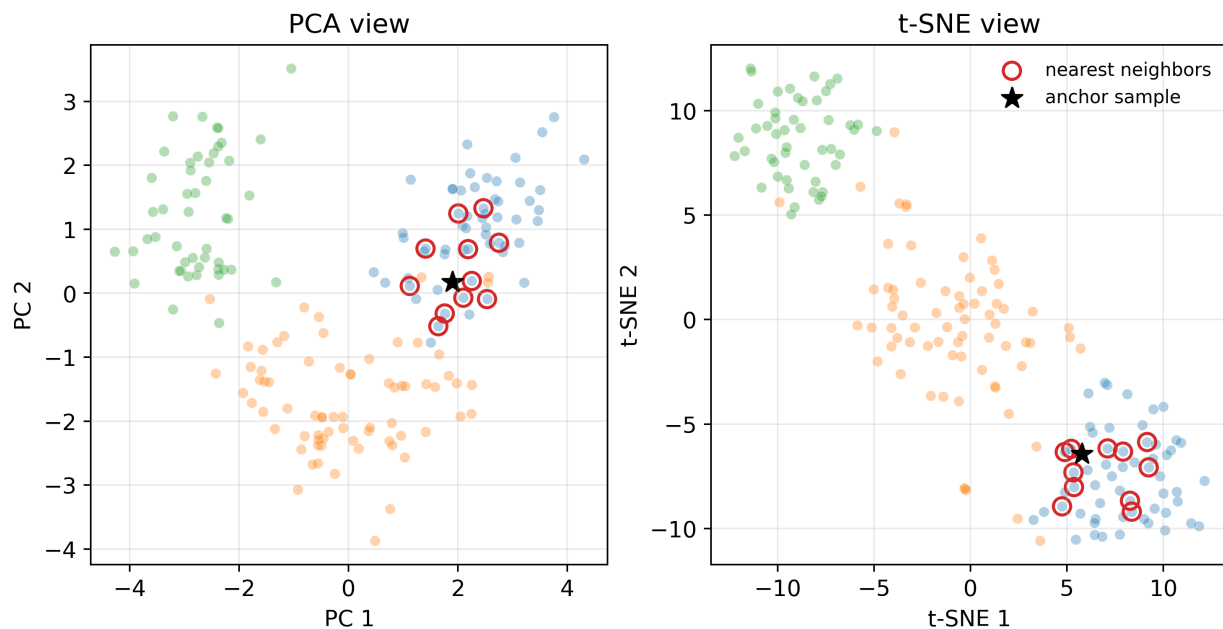


Figure 9.13: Wine data viewed with PCA and t-SNE. PCA preserves global linear variance, while t-SNE emphasizes local neighborhood structure.

A simplified view of t-SNE is as follows. In the original high-dimensional space, define a probability that point i chooses point j as a neighbor. Points that are close have larger probabilities. In the low-dimensional space, define a second set of neighbor probabilities. t-SNE then moves the low-dimensional points to make those two probability distributions similar.

The high-dimensional conditional probability is often written as

$$p_{j|i} = \frac{\exp\left(-\left\|\vec{x}^{(i)} - \vec{x}^{(j)}\right\|_2^2 / (2\sigma_i^2)\right)}{\sum_{k \neq i} \exp\left(-\left\|\vec{x}^{(i)} - \vec{x}^{(k)}\right\|_2^2 / (2\sigma_i^2)\right)}. \quad (9.21)$$

The bandwidth σ_i is chosen so that each point has a roughly specified number of effective neighbors. This number is controlled by a parameter called perplexity.

In the low-dimensional map, t-SNE uses heavy-tailed similarities. If $\vec{z}^{(i)}$ and $\vec{z}^{(j)}$ are embedded points, a common low-dimensional similarity is

$$q_{ij} = \frac{\left(1 + \left\|\vec{z}^{(i)} - \vec{z}^{(j)}\right\|_2^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \left\|\vec{z}^{(k)} - \vec{z}^{(l)}\right\|_2^2\right)^{-1}}. \quad (9.22)$$

The t-SNE objective minimizes a mismatch between high-dimensional and low-dimensional neighborhood probabilities:

$$C = \sum_i \sum_j p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right). \quad (9.23)$$

This is a Kullback–Leibler divergence. The details are less important than the interpretation: t-SNE tries to keep neighbors together.

Important t-SNE settings include:

- **Perplexity:** controls the effective neighborhood size.
- **Learning rate:** controls the optimization step size.
- **Initialization:** different starts can produce different layouts.
- **Random seed:** the same data can produce different plots if the seed changes.

Figure 9.14 shows the effect of perplexity on a synthetic data set. The same data can look different when the neighborhood scale changes.

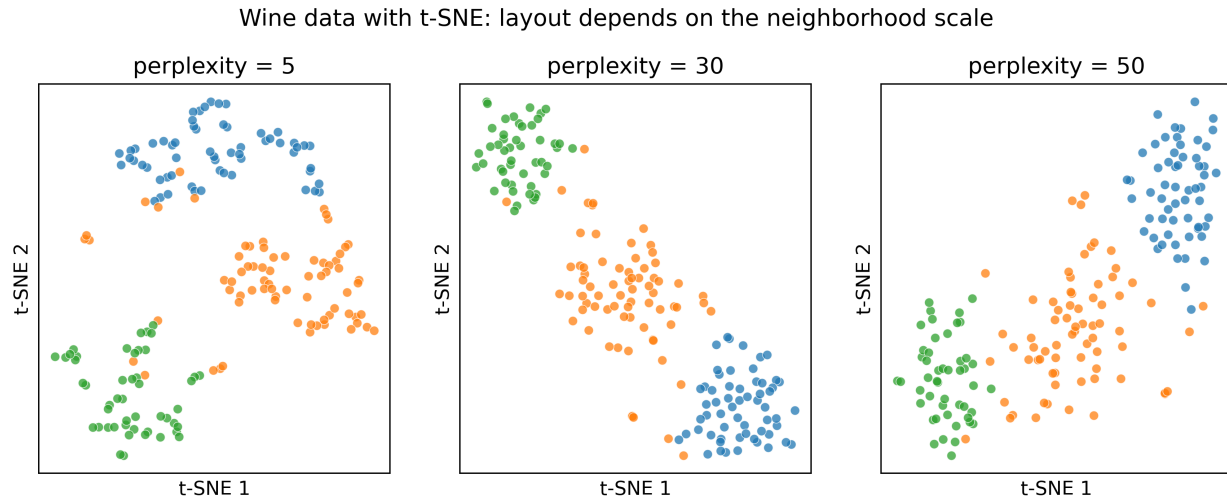


Figure 9.14: Effect of perplexity in t-SNE. Perplexity changes the effective neighborhood size, so visual layouts should be interpreted as exploratory summaries rather than exact geometric maps.

Example 9.3 t-SNE for inspection features. An engineer may extract feature vectors from images of manufactured parts and then plot those features with t-SNE. If images with similar defect patterns form neighborhoods, the plot can help organize inspection data. The plot should be used for exploration, not as a calibrated measurement of distance between defects.

9.4 Isometric Mapping (Isomap)

Isometric mapping, or Isomap, is another nonlinear dimensionality-reduction method included in `scikit-learn`. It is useful as a basic graph-based method because it extends classical multidimensional scaling with neighborhood information. Instead of measuring distance only by straight-line Euclidean distance in the original feature space, Isomap builds a nearest-neighbor graph and estimates distances along that graph.

A simplified Isomap workflow is:

1. choose a distance metric in the original space;
2. connect each sample to nearby samples;
3. estimate shortest-path distances along the neighbor graph;
4. find low-dimensional coordinates that preserve those graph distances.

This can help when the data lie on a curved low-dimensional surface. Figure 9.15 shows the classic idea. A rolled surface is nonlinear in the original three-dimensional coordinates. Isomap tries to unroll the surface by preserving distances along the neighborhood graph.

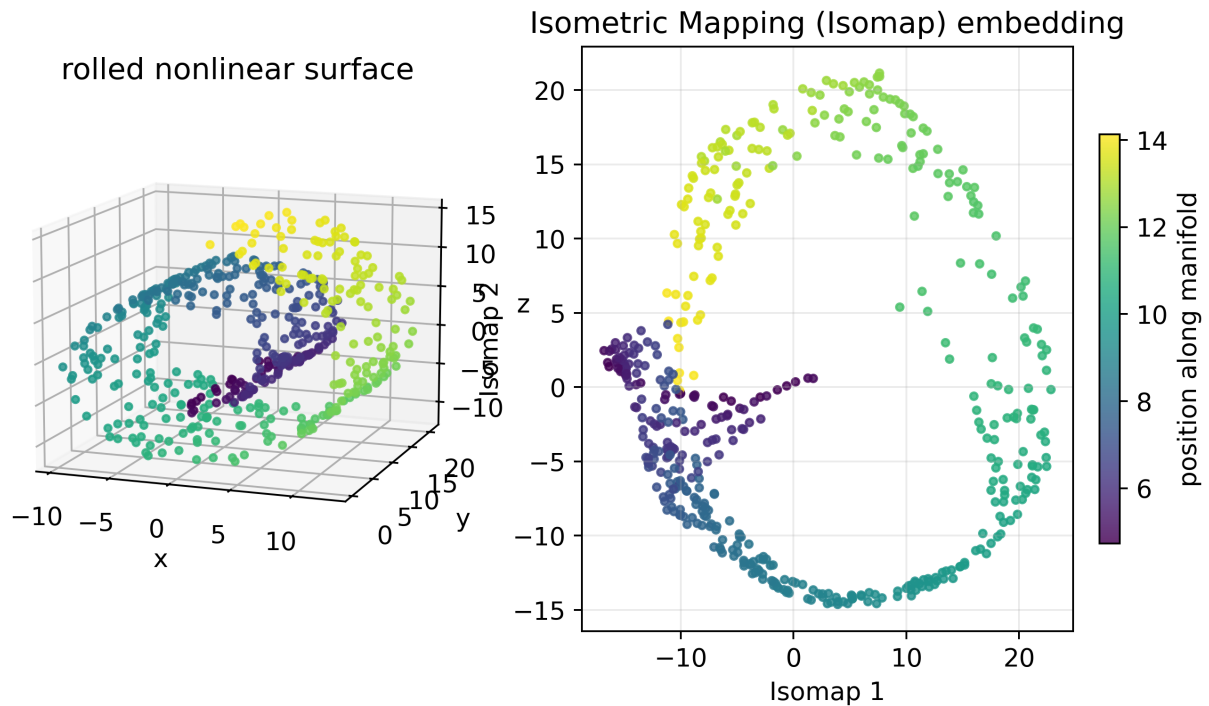


Figure 9.15: Isomap on a synthetic rolled surface. The method builds a neighborhood graph and finds coordinates that preserve distances along the graph, which can reveal curved low-dimensional structure.

Isomap is still an exploratory method. Its results depend on the number of neighbors and on whether the graph captures a meaningful manifold. If too few neighbors are used, the graph can break apart. If too many neighbors are used, the graph can behave more like ordinary Euclidean geometry.

Figure 9.16 compares PCA, t-SNE, and Isomap on the standardized Wine data. The methods give different views because they preserve different types of structure.

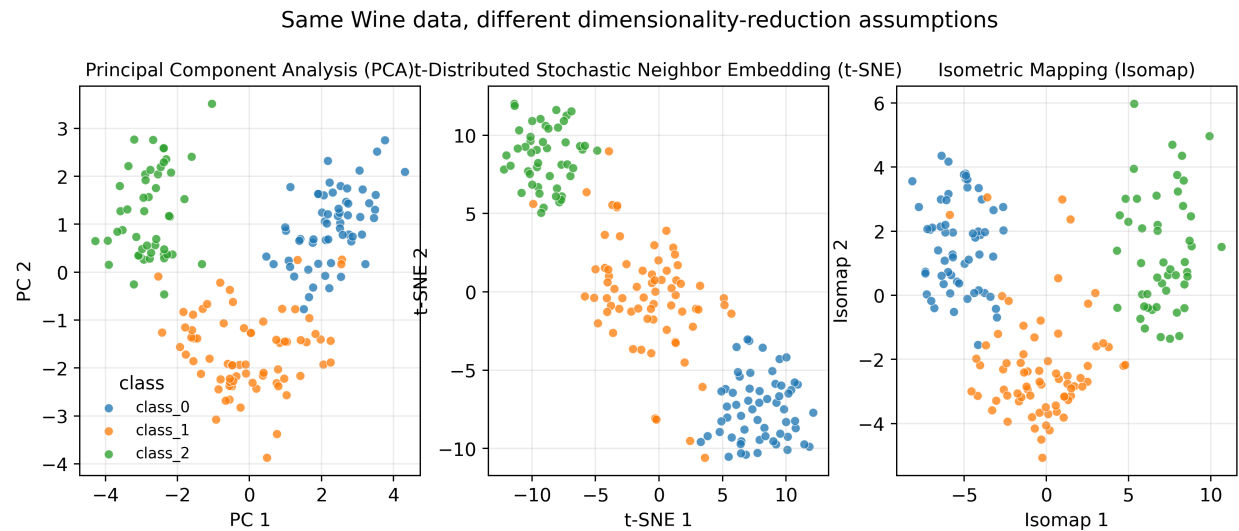


Figure 9.16: The same standardized Wine data viewed with PCA, t-SNE, and Isomap. PCA preserves global linear variance, t-SNE emphasizes local neighborhoods, and Isomap attempts to preserve graph-based geodesic distances.

Table 9.2 summarizes the methods discussed in this chapter.

Table 9.2: Practical comparison of dimensionality-reduction methods used in this chapter.

Method	Preserves	Typical use	Main caution
PCA	global linear variance	preprocessing, denoising, compression, baseline visualization	linear only; sensitive to feature scaling
t-SNE	local neighborhoods	exploratory two-dimensional visualization	global distances and cluster sizes can mislead
Isomap	neighborhood-graph distances	nonlinear visualization and manifold exploration	depends strongly on neighborhood graph quality

For an engineering workflow, a sensible sequence is to start with PCA. It is fast, deterministic, and interpretable. Then use t-SNE or Isomap for exploratory visualization if PCA does not reveal useful visual structure. Hard engineering claims should be validated with a task-specific model, not with a two-dimensional plot alone.

9.5 Practical Workflow

A clean dimensionality-reduction workflow should separate model fitting from evaluation. Figure 9.17 summarizes the main steps.

Practical dimensionality-reduction workflow



Use reduced coordinates only after checking what structure the method preserves.

Figure 9.17: Dimensionality-reduction workflow for engineering data. Scaling and PCA should be fit on the training data and then reused for validation, test, and future samples.

The following checklist is a good default.

1. **Define the goal.** Decide whether the reduced coordinates are for visualization, compression, denoising, anomaly detection, or downstream prediction.
2. **Split the data.** Keep training, validation, and test data separate when a later model will be evaluated.
3. **Scale using training data.** Estimate means and standard deviations only from the training set.
4. **Fit the reducer.** Fit PCA or another dimensionality-reduction method on the appropriate data subset.
5. **Transform or embed.** Compute reduced coordinates.
6. **Inspect.** Check plots, explained variance, loadings, residuals, clusters, and known operating conditions.
7. **Validate.** Measure performance on the actual engineering task.

For PCA, transforming a new sample is straightforward:

$$\vec{z}_{\text{new}} = \mathbf{U}_r^\top (\vec{x}_{\text{new}} - \vec{\bar{x}}_{\text{train}}). \quad (9.24)$$

If standardization was used, the new sample must be standardized with the training means and training standard deviations before projection.

For t-SNE and Isomap, the reduced coordinates are usually treated as visualization outputs. They are excellent for exploring data structure, but they are not always the right choice for a production preprocessing step. PCA is the safer first choice when future samples must be transformed in a repeatable engineering workflow.

9.6 Implementation in `scikit-learn`

For most engineering projects, dimensionality reduction should be implemented as part of a reproducible preprocessing pipeline. A PCA pipeline for the Wine example has the form

`StandardScaler()` $\rightarrow$ `PCA(n_components=r)` $\rightarrow$ `LogisticRegression()`. (9.25)

The scaler estimates the training feature means and standard deviations. PCA estimates the principal-component directions from the scaled training data. The classifier is then trained on the PCA scores.

A key implementation detail is the difference between `fit`, `transform`, and `fit_transform`. The method `fit` estimates parameters from data. For PCA, those parameters include the feature means and loading vectors. The method `transform` applies the already fitted transformation to data. The method `fit_transform` does both at once and is usually used only on the training data.

Table 9.3: Common `scikit-learn` classes for the methods used in this chapter.

Method	Typical class
Feature standardization	<code>sklearn.preprocessing.StandardScaler</code>
Principal component analysis	<code>sklearn.decomposition.PCA</code>
t-distributed stochastic neighbor embedding	<code>sklearn.manifold.TSNE</code>
Isometric mapping	<code>sklearn.manifold.Isomap</code>
Pipeline construction	<code>sklearn.pipeline.Pipeline</code>

The usual pattern for supervised evaluation is:

1. split the data into training and test sets;
2. build a pipeline containing scaling, dimensionality reduction, and the downstream model;
3. fit the pipeline on the training data only;
4. evaluate the full pipeline on validation or test data;
5. choose the number of components using validation performance and engineering judgment.

For t-SNE and Isomap, the output is usually treated as an exploratory visualization. PCA is usually the first choice when the reduced coordinates must be reused for future samples in a production workflow.

9.7 Common Mistakes

Dimensionality reduction is easy to run and easy to overinterpret. Common mistakes include:

- fitting scaling or PCA before the train-test split;
- treating t-SNE cluster distances as physical distances;

- choosing the number of PCA components only from a fixed variance rule;
- ignoring units and feature scaling;
- using a pretty two-dimensional plot as proof of model performance;
- forgetting that rare but important events may lie in low-variance directions;
- assuming that nonlinear embeddings preserve all structure in the original data.

The reduced coordinates should be evaluated in the context of the engineering task. If the task is visualization, the plot should be stable and interpretable. If the task is preprocessing, the downstream model should improve or become simpler. If the task is anomaly detection, the method should be tested on known anomalies or realistic fault scenarios.

9.8 Summary

Dimensionality reduction provides a bridge between raw high-dimensional data and interpretable, efficient machine-learning workflows. PCA gives a linear, variance-based coordinate system and is the main baseline. PCA uses the covariance matrix of centered data, finds the eigenvectors of that covariance matrix, and uses the eigenvalues to determine explained variance. Projection, reconstruction, and residuals make PCA useful for visualization, denoising, compression, preprocessing, and monitoring.

t-SNE gives a nonlinear local-neighborhood visualization. It is useful for exploring high-dimensional data, but global distances and cluster sizes should be interpreted cautiously. Isomap introduces graph-based manifold learning within `scikit-learn`. It can reveal nonlinear neighborhood structure, but it depends on how the neighborhood graph is built.

The main engineering lesson is simple: use dimensionality reduction to summarize data, not to hide assumptions. Always record how the reducer was fit, how the data were scaled, how many dimensions were kept, and how the reduced coordinates were evaluated.